

Disciplina: Qualidade de Software e Testes

Professor: Leonardo Cardia

MELHORIA REALIZADA

Gilded Rose — Modernização de Sistema Legado

2026

MELHORIA REALIZADA

Descrição das alterações feitas no sistema Gilded Rose e justificativa técnica de cada uma, em complemento ao diagnóstico (docs/diagnostico_tecnico.md) e ao plano de contingência (docs/plano_contingencia.md).

1. Extração de Item e GildedRose para src/, sem alterar lógica

O que foi feito: código movido de `legacy/gilded_rose.py` para `src/domain/item.py` (classe `Item`, inalterada) e `src/application/gilded_rose.py` (classe `GildedRose`, lógica idêntica ao original neste primeiro passo).

Por quê: estabelecer a estrutura de pastas da arquitetura limpa antes de qualquer mudança de comportamento, permitindo confirmar — com os testes de caracterização — que a simples mudança de local não alterou nada.

2. Substituição dos condicionais aninhados por Strategy (src/domain/item_updaters.py)

O que foi feito: cada tipo de item (comum, Aged Brie, Sulfuras, Backstage Passes) passou a ter sua própria classe `*Updater` com um único método `update(item)`. `GildedRose.update_quality` foi reduzido a um laço que resolve a estratégia certa por nome (`resolve_updater`) e delega.

Por quê: resolve diretamente os code smells 1, 2, 3 e 4 do diagnóstico técnico (condicionais aninhados, comparação de string repetida, responsabilidades misturadas, regra de Backstage acoplada à regra de Aged Brie). Cada regra agora pode ser lida, entendida e testada isoladamente, sem precisar instanciar o sistema inteiro.

3. Centralização dos limites de quality (MIN_QUALITY, MAX_QUALITY)

O que foi feito: as constantes 0 e 50, antes repetidas como literais em 4 e 3 pontos do código respectivamente, foram centralizadas em `src/domain/item_updaters.py` e aplicadas via `max()/min()`.

Por quê: resolve o code smell 5 do diagnóstico (duplicação de regra de limite) — alterar o limite de qualidade no futuro exige mudar em um único lugar.

4. Centralização dos nomes de item (src/domain/item_names.py)

O que foi feito: as strings "Aged Brie", "Sulfuras, Hand of Ragnaros" e "Backstage passes to the TAFKAL80ETC concert" passaram a ser constantes nomeadas, reutilizadas tanto pela lógica de domínio quanto pelos testes.

Por quê: elimina o risco de divergência entre o nome usado na lógica e o nome usado no teste (erro de digitação silencioso), e documenta a intenção de cada string.

5. Implementação da regra de itens Conjurados (ConjuredItemUpdater)

O que foi feito: nova estratégia que degrada quality em 2 por dia antes do vencimento e 4 por dia depois, respeitando o limite mínimo de 0. Identificação por prefixo (item_names.is_conjured, `check name.startswith("Conjured")`) em vez de nome exato, cobrindo a categoria inteira, não só "Conjured Mana Cake".

Por quê: corrige o bug confirmado no legado (item já citado no cenário de demonstração, mas tratado como item comum) e atende RF11 (docs/requisitos.md). A escolha por prefixo em vez de nome exato segue literalmente o enunciado, que pede suporte à "categoria de itens Conjurados", não a um item específico.

6. Testes em duas camadas

O que foi feito: tests/test_caracterizacao.py (testes de ponta a ponta via GildedRose, herdados da rede de segurança original) e tests/test_item_updaters.py (testes unitários de cada *Updater isoladamente, sem instanciar GildedRose).

Por quê: demonstra concretamente a melhoria de testabilidade (RNF04) — algo impossível de fazer de forma isolada no código legado.

Evidência de que nada quebrou

docs/evidencias/diff_antes_depois.txt mostra a única diferença entre a execução do legado e do código refatorado ao longo de 30 dias: o item Conjured, que mudou de propósito (era bug, agora é regra correta). Todos os outros itens — comum, Aged Brie, Sulfuras, Backstage — produzem saída idêntica.